

glm-cli-plugin-cc

Give GLM its own agentic CLI — then wrap it as a Claude Code engine

Design Specification | v1 | Swimlane 2 (2.1 CLI + 2.2 plugin) | Author: Luan Moreno | Status: DRAFT for build

This is a **self-contained build brief** for a fresh agent session. Unlike Codex and Kimi, GLM ships **no agentic CLI** — it is a model API. So this swimlane is deliberately split into two atomic, focused units: **2.1** build a real glm coding CLI (an agent loop pointed at GLM's Anthropic-compatible endpoint), and **2.2** wrap it in a thin plugin identical in shape to codex-plugin-cc. 2.2 depends on 2.1; 2.1 depends on nothing.

The decisive fact (verified via Context7 MCP against z.ai docs)

GLM exposes an **Anthropic-compatible endpoint** at `https://api.z.ai/api/anthropic` (auth via `ANTHROPIC_AUTH_TOKEN`), explicitly built so Claude-Code-style clients can run on GLM models (e.g. `glm-4.6`). It also offers an OpenAI-compatible chat endpoint at `https://api.z.ai/api/paas/v4/chat/completions`. **That protocol compatibility is what makes a GLM CLI a build, not a research project.**

1 Why the swimlane splits into 2.1 and 2.2

Codex and Kimi are **full agentic CLIs** — they own a tool loop (read / edit / shell), a sandbox, and sessions. The plugins merely *wrap the agent*. GLM has no such CLI. If we skipped straight to a plugin, we'd be forced to drive GLM through `claude -p` pointed at its Anthropic endpoint — a borrowed tool loop. That works, but it is a hack: GLM would masquerade as Claude Code. You cannot cleanly “offer” that as a product.

So we do it honestly: **build the CLI first**, then the plugin is a mechanical port of Swimlane 1.

Unit	Deliverable	Depends on	Nature
2.1	glm CLI — agentic coding tool (its own binary)	■ ■ ■	Medium build; the real work
2.2	glm-plugin-cc — wraps <code>glm --print`</code>	2.1	Thin port of codex-plugin-cc

Forced ordering — a clean DAG

2.2 blocked-by 2.1; 2.1 has no deps. Swimlane 1 (codex) runs fully in parallel with 2.1. This maps 1:1 onto the Task-Spec model — three atomic units, one real `depends_on` edge — so the fleet orchestrator can schedule them directly.

2 Unit 2.1 — the glm CLI

Goal: a coding CLI that behaves like `codex / kimi` — a real agent loop over GLM models — and is **plug-compatible** with the crank spine (`--print`, `--json`, `--work-dir`, `--resume`).

2.1.1 Build vs. fork — settle by MCP review, not by guessing

The single biggest unknown in the whole plan. Do NOT assume; run a grounded MCP pass first.
Deliverable: a recommendation with evidence.

- Probe the endpoint** — use Context7 / ref / exa MCPs (and a tiny live call) to confirm what `api.z.ai/api/anthropic` actually supports: *tool-use* (function calling), *streaming*, and *session/resume* semantics a crank needs.

- **Survey OSS agentic CLIs** that already speak the Anthropic protocol, scored on: license (MIT/Apache preferred), tool-loop maturity, and how cleanly they accept a custom `base_url` + token.
- **Decide:** fork a mature OSS CLI (fast to a working loop, inherits its architecture/license) vs. build thin on the Anthropic SDK (full control, plug-compatible, no upstream baggage).

Option	Pros	Cons	Pick when
Fork OSS Anthropic-protocol CLI	Fastest to a working agent loop; battle-tested tool handling	Inherits architecture + license; adaptation surface	A clean MIT/Apache CLI points at a custom <code>base_url</code> with little friction
Build thin on Anthropic SDK	Full control; exact flag parity; no upstream drift	You implement the tool loop (read/edit/shell) yourself	No OSS CLI adapts cleanly, or you want a product you fully own
Spike then decide	Evidence-driven; de-risks the whole swimlane	20-min upfront cost	DEFAULT — always do this first

2.1.2 Required flag surface (plug-compatibility contract)

Whatever the build path, the CLI **MUST** expose these so the crank spine drives it unchanged — the same way it drives `kimi --print` and `codex exec`:

Flag	Meaning	Mirrors
<code>-p / --print "<prompt>"</code>	Headless: run once, non-interactive, print result	<code>kimi --print</code> / <code>codex exec</code>
<code>--json (stream)</code>	Emit JSONL events to stdout (tool calls, thinking, final)	<code>codex --json</code>
<code>--work-dir / -C <dir></code>	Working root (isolated worktree for parallel cranks)	<code>kimi --work-dir</code> / <code>codex --cd</code>
<code>-o / --output-last-message <file></code>	Write final assistant message to a file	<code>codex -o</code>
<code>--model <id></code>	GLM model (glm-4.6 default; others per z.ai)	both
<code>--resume [--last]</code>	Continue a prior session by id / most recent	<code>codex exec resume</code>
<code>--yolo / --auto-approve</code>	Non-interactive write + shell (crank needs this)	<code>kimi --yolo</code>
<code>--agent-file <path></code>	System prompt / tool policy for the loop (optional)	<code>kimi --agent-file</code>

2.1.3 Auth & config

```
# The CLI reads these (env or ~/.glm/config):
GLM_API_KEY           = <your z.ai api key>           # required
GLM_BASE_URL          = https://api.z.ai/api/anthropic # Anthropic-compatible loop
GLM_MODEL             = glm-4.6                      # default model

# Internally the agent loop calls the Anthropic-shaped endpoint:
# POST https://api.z.ai/api/anthropic/v1/messages (tool-use + streaming)
# OpenAI-compatible fallback for simple calls (review/ask, no tools):
# POST https://api.z.ai/api/paas/v4/chat/completions model=glm-4.6
```

Never hardcode the key. Read from env; support .env autoloading and (optionally) Infisical via the repo's infisical-manager.

2.1.4 The agent loop (what 'baked in' means)

A coding CLI is a loop: send messages + tool schemas → model returns tool calls → execute tools → feed results back → repeat until the model emits a final answer. The minimum viable tool set for cranking Task-Specs:

- **ReadFile / Glob / Grep** — navigate the repo (read-only).
- **WriteFile / Edit** — produce the diff (write-capable, scoped to --work-dir).
- **Shell** — run the Task-Spec eval / build (sandboxed; the crank's done-condition).
- **Session persistence** — write JSONL transcript + meta so --resume and the plugin's status/result/watch work.

Emit the same JSONL shape the spine already parses

Kimi's watcher keys on tool-call names (ReadFile/WriteFile/Edit/Shell) and roles (assistant/think). If the GLM CLI emits the same event shape, the plugin's watch/status progress stream works with zero changes. Design the JSONL to match.

3 Unit 2.2 — glm-plugin-cc (the wrapper)

Once 2.1 is plug-compatible, this is a **near-copy of codex-plugin-cc** with the driver swapped from codex exec to glm --print. Same commands, same spine, same exit codes.

Command	Purpose	Ported from
/glm:crank <task.md>	Execute one Task-Spec; write-capable; eval-gated	codex:crank
/glm:crank-next	Highest-priority ready task off the depends_on DAG	codex:crank-next
/glm:crank-batch <glob>	Dependency-respecting waves (orchestrate.mjs)	codex:crank-batch
/glm:review	Peer-review a diff (chat endpoint; read-only)	kimi:review pattern
/glm:challenge	Adversarial critique of design/assumptions	kimi:challenge
/glm:ask "<q>"	One-shot GLM question (chat completions)	new (thin)
/glm:status result cancel setup	Session mgmt + CLI/auth verification	codex:*

Driver swap, concretely: the broker's dispatch calls glm.mjs (a copy of codex.mjs that spawns glm instead of codex exec). Everything else — orchestrate, preflight, eval gate, diff capture, session state — is identical.

File manifest (delta from codex-plugin-cc)

```
glm-plugin-cc/
.claude-plugin/plugin.json      # name: glm-crank
commands/  glm:crank.md  glm:crank-next.md  glm:crank-batch.md
           glm:review.md  glm:challenge.md  glm:ask.md
           glm:status.md  glm:result.md    glm:cancel.md    glm:setup.md
scripts/
broker.mjs                                # dispatcher (spine, verbatim)
lib/
  glm.mjs                                â■ # spawns `glm --print` (copy of codex.mjs)
  glm-chat.mjs                          â■ # POST /paas/v4/chat/completions (review/ask)
  orchestrate.mjs  preflight.mjs  git.mjs  state.mjs  # spine (verbatim)
README.md  CHANGELOG.md  LICENSE (MIT)
```

4 GLM's two API surfaces — and which to use where

Surface	Endpoint	Used by	Why
Anthropic-compatible	api.z.ai/api/anthropic	the CLI agent loop (2.1) → /glm:crank	Supports tool-use + streaming needed for an agent loop
OpenAI-compatible chat	api.z.ai/api/paas/v4/chat/completions	/glm:review, /glm:challenge, /glm:ask	Single-shot completion; no tool loop needed; simplest for verdicts

Verified request shapes (from z.ai docs via MCP)

```
# Chat completion (review / challenge / ask) â OpenAI-compatible
curl -X POST https://api.z.ai/api/paas/v4/chat/completions \
-H "Authorization: Bearer $GLM_API_KEY" -H 'Content-Type: application/json' \
-d '{ "model": "glm-4.6",
      "messages": [{"role": "user", "content": "<diff + review rubric>"}],
      "thinking": {"type": "enabled"}, "max_tokens": 4096, "temperature": 0.6 }'

# Agent loop (crank) â Anthropic-compatible; the glm CLI drives this internally
export ANTHROPIC_BASE_URL=https://api.z.ai/api/anthropic
export ANTHROPIC_AUTH_TOKEN=$GLM_API_KEY
# glm --print "<task body>" --work-dir . --json -o last.txt --yolo
```

glm-4.6 confirmed as a coding model id; `thinking:{type:enabled}` toggles reasoning on the chat surface.

5 How all three engines converge

The end state after Swimlane 1 + 2.1 + 2.2: three interchangeable engines, one spine, one gate.

```
tasks/T-*.md â●●â●-â●● kimi --print (exists)
â●●â●● codex exec (Swimlane 1)
â●●â●● glm --print â●● api.z.ai/api/anthropic (Swimlane 2)
every one gated by the SAME Task-Spec eval â●● R6 holds
```

Capability	kimimaker	codex	gpt-4o
Review	yes	yes	yes (chat)
Adversarial	yes	yes	yes (chat)
Execute/crank	yes	yes (SL1)	yes (SL2, via CLI)

Net capability added to Claude: it orchestrates, and dispatches **review / adversarial / execute** to whichever engine fits — on cost, speed, or a genuine **second opinion from a different model family** (the adversarial value is real precisely because GLM $\neq$ Codex $\neq$ Claude). `task-runner --agent {claude|codex|kimi|glm}` becomes true.

6 Phased build plan

Phase	Unit	Deliverable	Acceptance gate
G0	2.1	MCP review: probe endpoint + survey OSS CLIs; write build-vs-fork ADR	ADR recommends a path with cited evidence
G1	2.1	glm CLI: agent loop (read/edit/shell) over api.z.ai/api/anthropic	`glm --print "list files"` returns a real answer; tool-use round-trips
G2	2.1	Plug-compat flags: --print --json --work-dir -o --resume --yolo	Emits JSONL matching the spine's watcher; -o writes final message
G3	2.1	Session persistence + resume	A cancelled run resumes with --last and continues
G4	2.2	glm-plugin-cc: port codex-plugin-cc; swap driver to glm.mjs	/glm:crank cranks a fixture Task-Spec to green; eval gate blocks buggy evals (exit 3)
G5	2.2	/glm:review /glm:challenge /glm:ask via chat endpoint	Review returns a structured verdict on a real diff
G6	2.2	Publish; wire task-runner --agent glm	fleet-orchestrator lists glm as a worker; batch runs in waves

Acceptance eval (2.1 plug-compatibility smoke)

```
set -euo pipefail
# CLI answers headlessly and writes the final message
glm --print "print the word READY and nothing else" --json -o /tmp/glm-last.txt
grep -q READY /tmp/glm-last.txt
# tool-use round-trips: ask it to read a file it must open
echo 'hello-from-fixture' > /tmp/fx.txt
glm --print "read /tmp/fx.txt and echo its contents" --work-dir /tmp | grep -q hello-from-fixture
```

7 Risks & open questions

Risk / question	Handling
Does api.z.ai/api/anthropic fully support tool-use + streaming?	Settle in G0 MCP review + a live probe BEFORE committing to build-vs-fork.
GLM API key management	Env GLM_API_KEY + .env autoload; optionally Infisical. Never hardcode.
Spine drift across three engines	Vendor-copy for v1; extract @owshq/crank-spine once all three are stable.
glm-4.6 vs newer model ids	Model is a --model flag + GLM_MODEL env; do not hardcode a single id.
Rate limits / cost on batch cranks	Respect --max-parallel; surface usage from the chat endpoint's usage block.
CLI masquerading as Claude Code (the hack we rejected)	We build a real glm binary (2.1) so the plugin wraps an honest CLI, not claude -p.

Non-goals for v1: multimodal (image/video) GLM features; a GLM MCP server; multi-repo cranks; anything the Task-Spec eval gate cannot machine-check.

End of glm-cli-plugin-cc v1 design spec. Companion: **codex-plugin-cc-v1.pdf** (Swimlane 1). Build order: Swimlane 1 and 2.1 in parallel; 2.2 after 2.1. All three engines share one crank spine and one eval gate — “eval passes = done” regardless of which model runs.